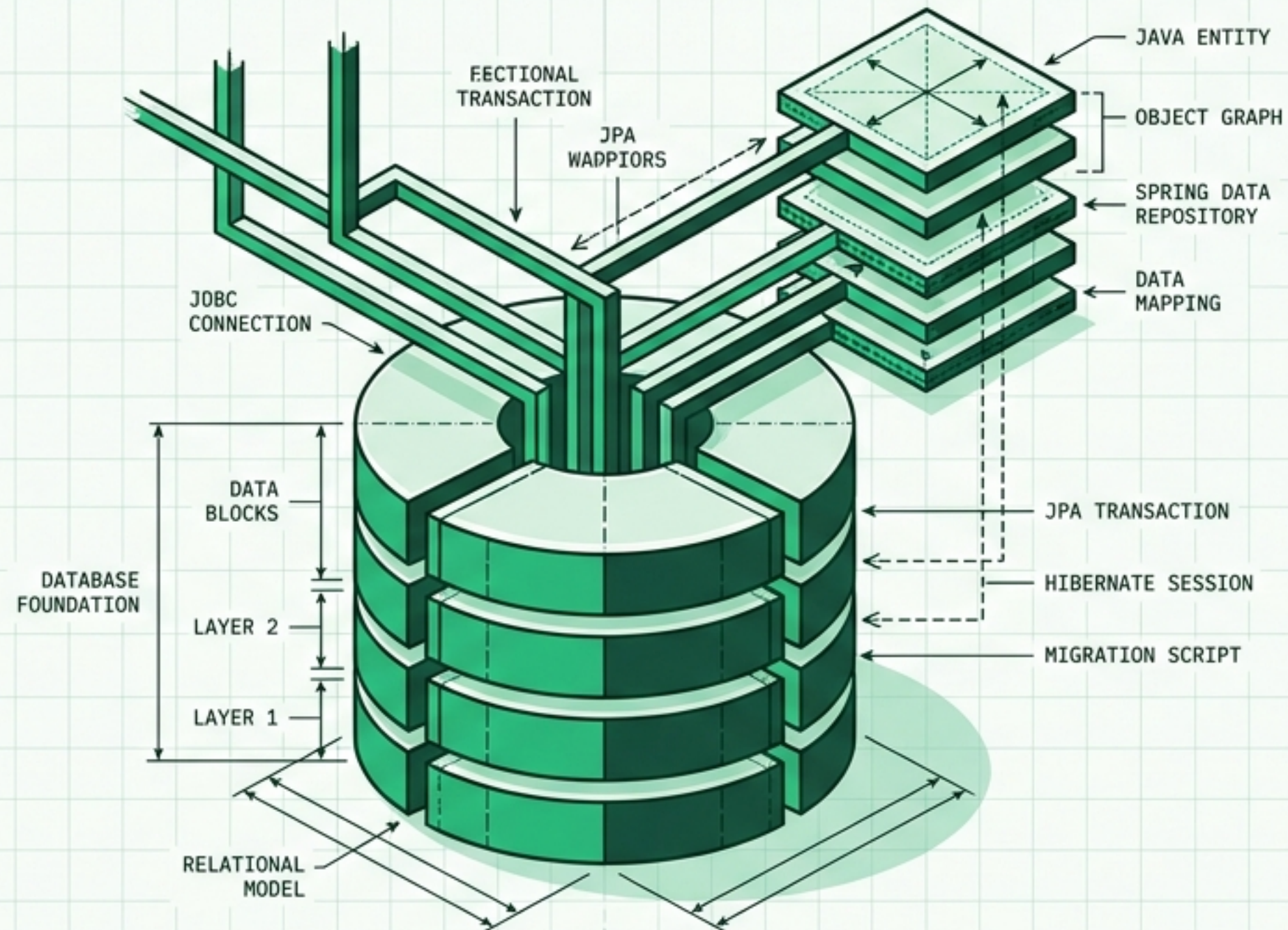
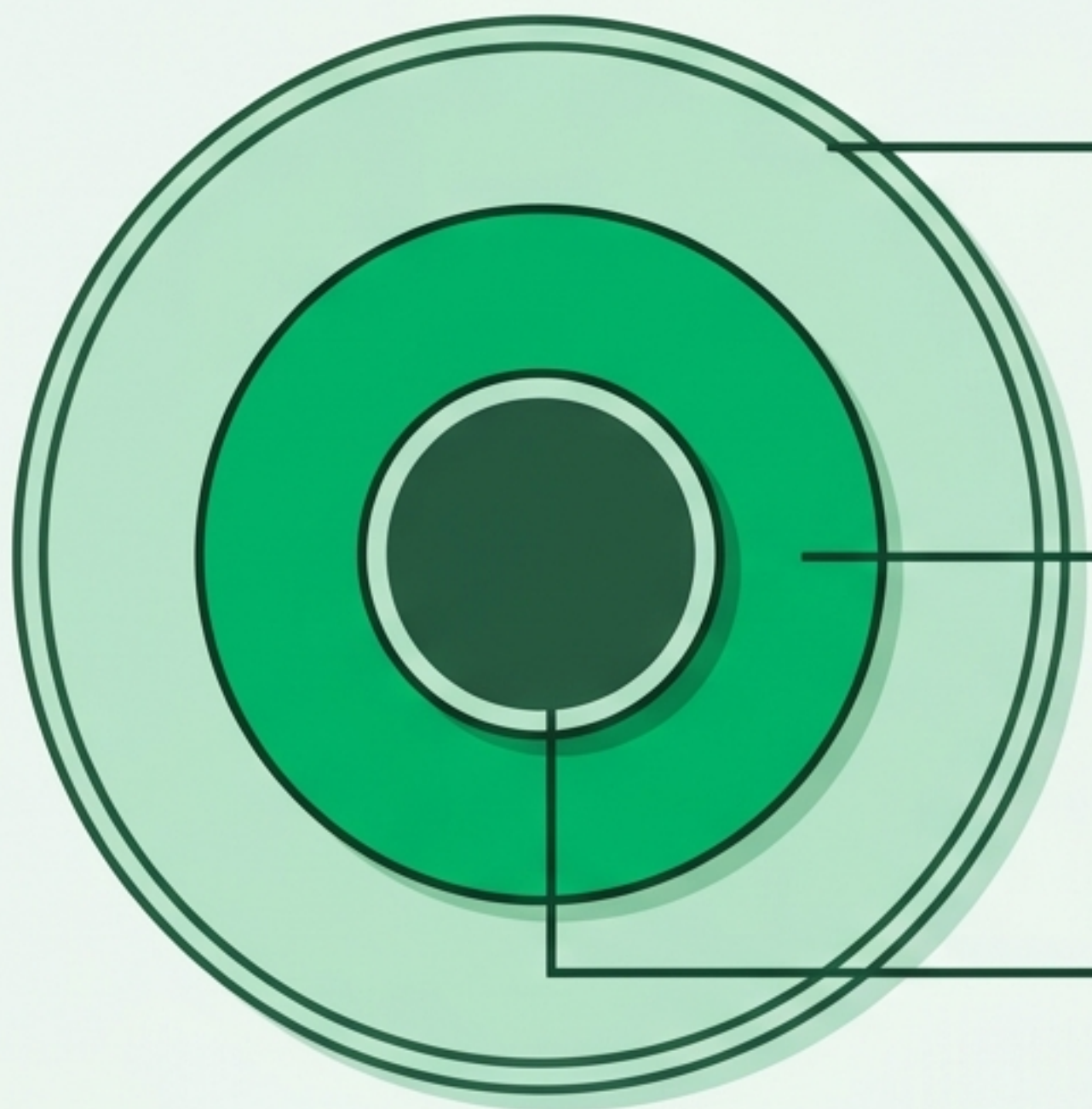


TALLER PRÁCTICO DE JAVA CON SPRING BOOT

Módulo 6: Acceso a Datos

Dominando Spring Data JPA, relaciones, transacciones y migraciones





Spring Data JPA

La abstracción final (La API que usas).
Genera implementaciones en tiempo de ejecución para evitar código repetitivo.

Hibernate

El motor / La implementación. Hace el trabajo pesado: convierte objetos a SQL y maneja el contexto de persistencia.

JPA (Java Persistence API)

El estándar / Especificación JSR. Define cómo mapear objetos a tablas (ORM) pero no hace el trabajo por sí solo.

Nota Arquitectónica:

JPA abstrae el SQL pero no lo elimina. Saber SQL sigue siendo esencial para optimizar (índices, planes de ejecución, joins).

Anatomía de una Entidad JPA

```
@Entity
@Table(name="producto")
public class Producto {

    @Id
    @GeneratedValue
    private Long id;

    @Column
    private String nombre;

    @CreationTimestamp
    private LocalDateTime creado_en;
}
```

Marca la clase y define la tabla destino.

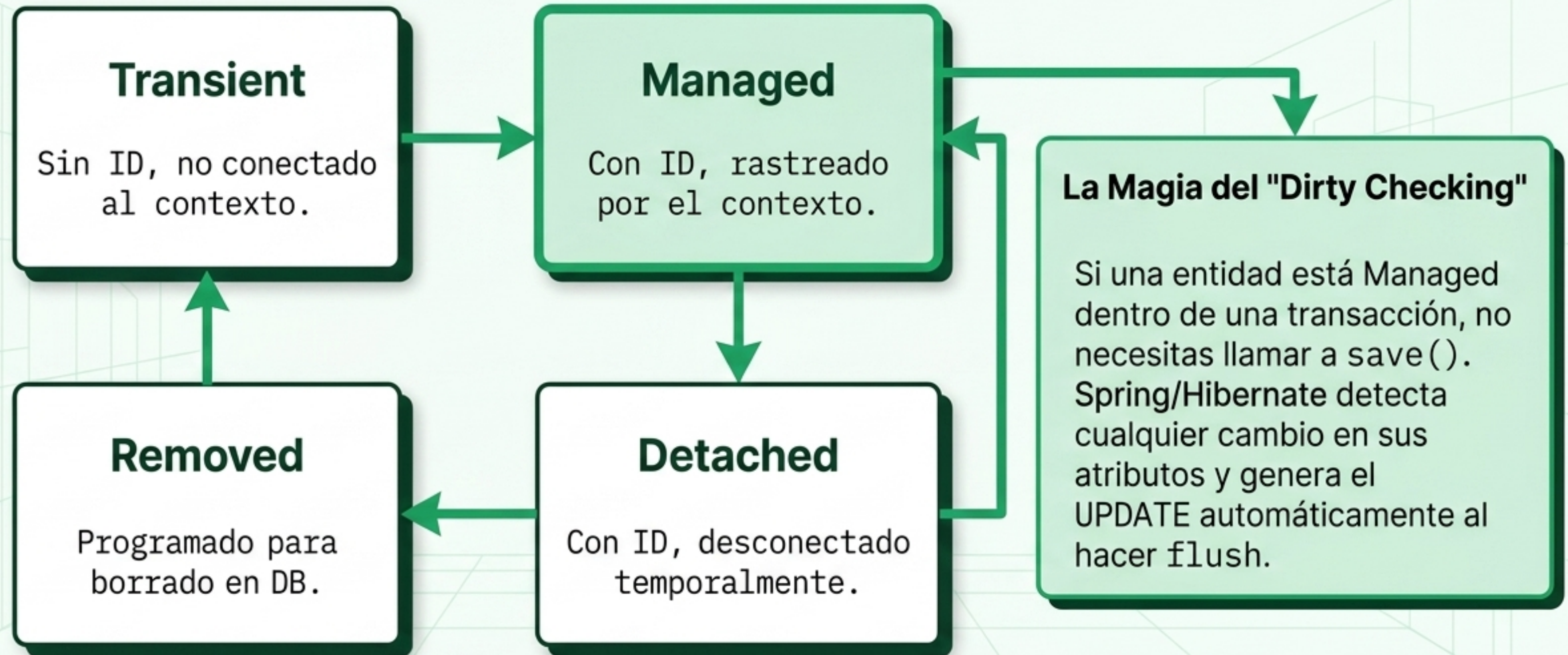
Define la Clave Primaria (PK) autogenerada.

Configuración específica (ej. nombre, nulabilidad).

Fecha automática gestionada por el framework.

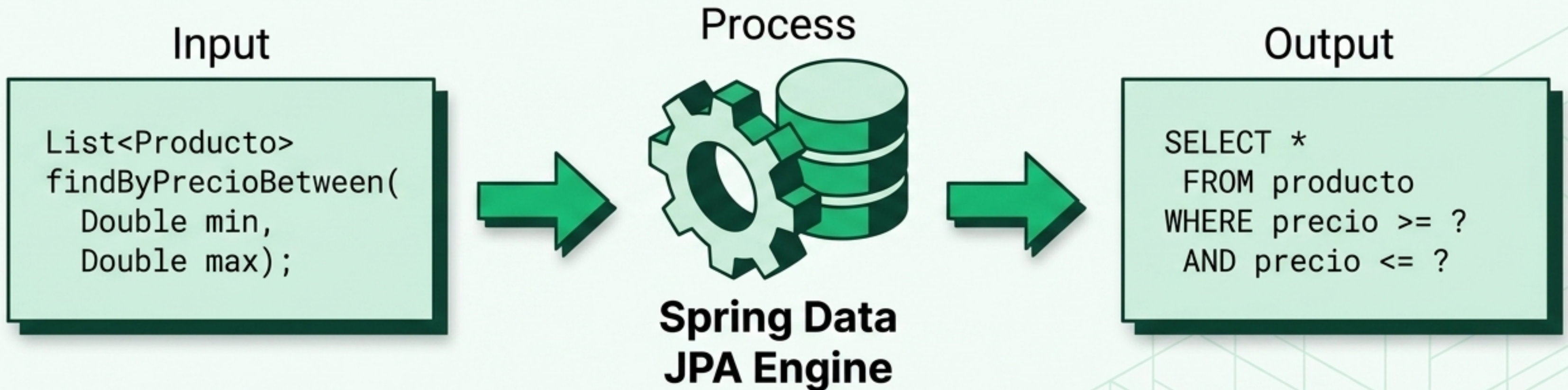
id	nombre	creado_en

El Ciclo de Vida de la Entidad (Persistence Context)



La Magia de Spring Data Repositories

Spring Data genera la implementación en tiempo de ejecución.

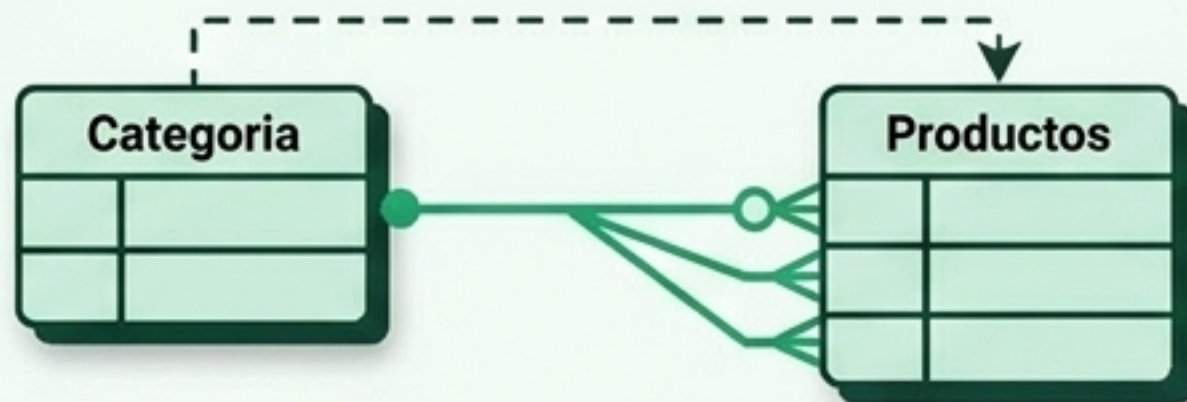


Consultas Derivadas	Consultas Personalizadas
Convenciones de nombres: findByNombre, findByCategoriaId	Usando @Query("SELECT p FROM Producto p...") cuando la convención no es suficiente.

Mapeo de Relaciones y Direccionalidad

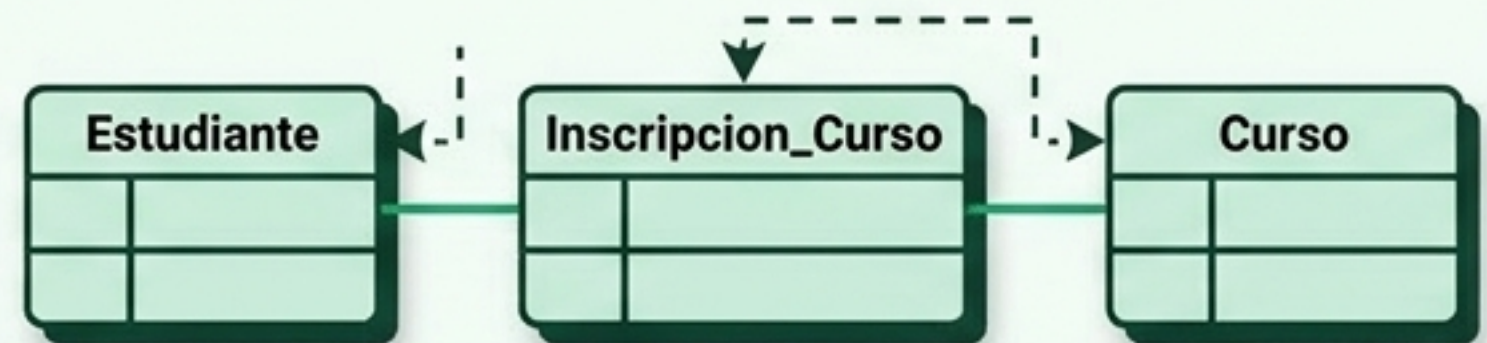
@ManyToOne / @OneToMany

Relación padre-hijo (Ej. Categoría -> Productos).



@ManyToMany

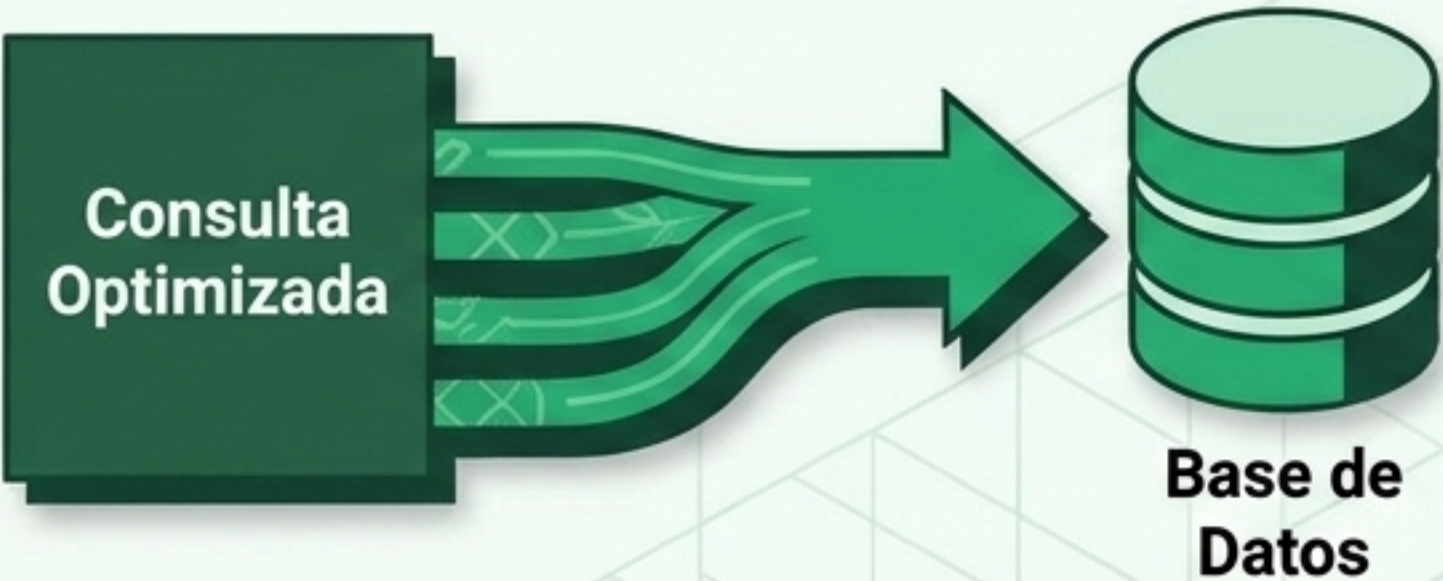
Relación compleja. Siempre requiere la creación de una tabla de unión intermedia.



El Atributo `mappedBy`:

Marca el lado inverso de la relación. Solo el lado dueño de la relación mantiene la Clave Foránea (Foreign Key). El lado mappedBy es estrictamente de solo lectura para el framework ORM.

Diagnosticando el Problema N+1

El Problema (1 + N)	La Solución (JOIN FETCH)
FetchType.LAZY (por defecto) 	Extracción Optimizada 
Cargar una lista y luego iterar sus hijos lanza 1 consulta inicial + 1 consulta adicional por cada hijo. Spring Data no te protege automáticamente.	Usa <code>@Query("SELECT p FROM Producto p JOIN FETCH p.categoria")</code> o <code>@EntityGraph(attributePaths = "categoria")</code> para forzar la carga en una sola consulta.

Fundamentos ACID en Transacciones

Definiendo la unidad de trabajo (Todo o nada)



Atomicidad

Todo o nada. Si falla un paso (ej. el dinero sale pero no llega), toda la operación se revierte.



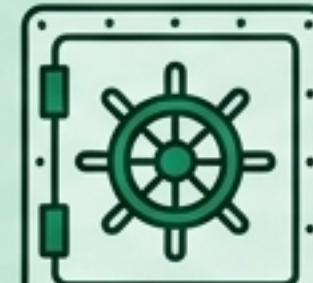
Consistencia

El estado siempre es válido. Las invariantes y reglas de negocio se mantienen al inicio y fin.



Aislamiento

Las transacciones concurrentes no se interfieren entre sí. Evita lecturas a medias.

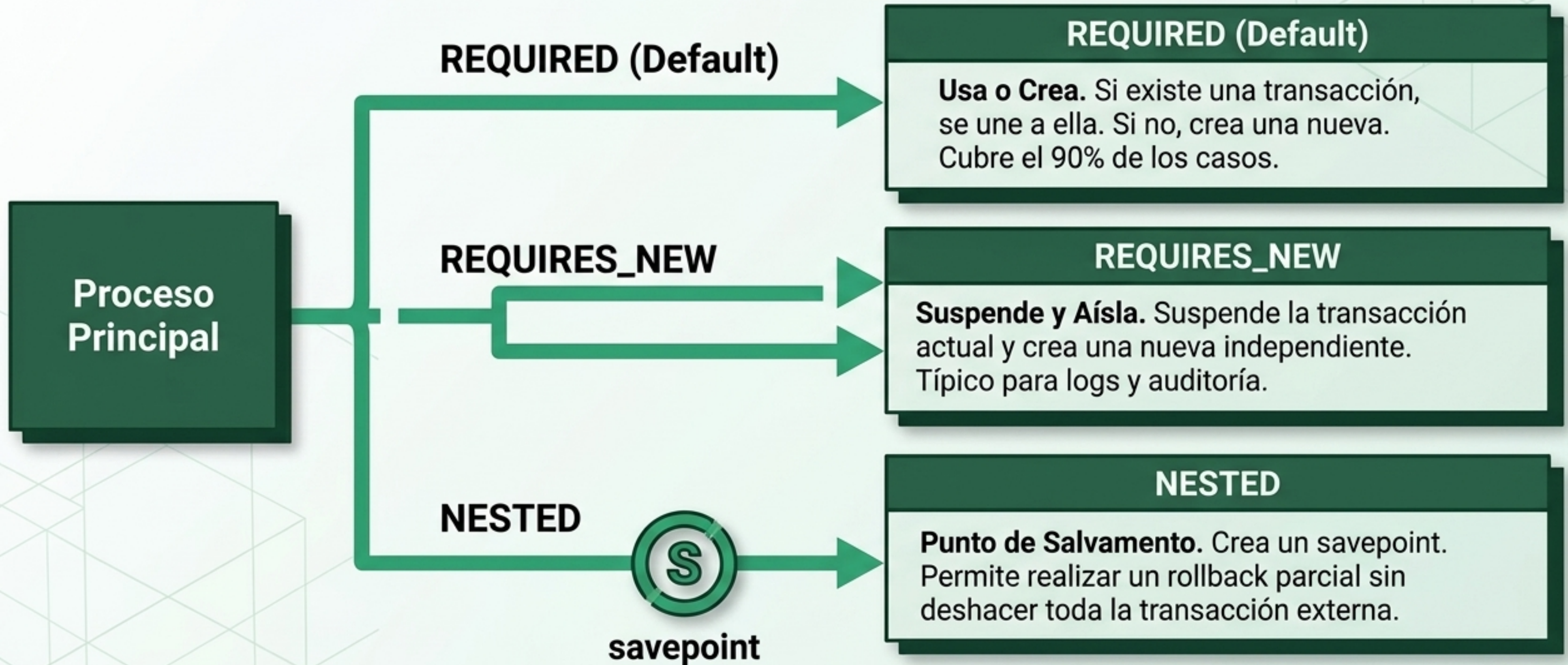


Durabilidad

Lo confirmado sobrevive. Si el sistema colapsa tras el commit, los datos no se pierden.

Matriz de Propagación (@Transactional)

Cómo reacciona Spring cuando una transacción llama a otra.



Concurrencia y Niveles de Aislamiento

El nivel define qué fenómenos de concurrencia se permiten a expensas del rendimiento.

Nivel de Aislamiento	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	✗	✗	✗
READ COMMITTED	✓	✗	✗
REPEATABLE READ	✓	✓	✗
SERIALIZABLE	✓	✓	✓

[Check] = Evitado (Seguro) [X] = Permitido (Vulnerable)

Evolución del Esquema con Flyway

Las migraciones aplican cambios secuenciales y registran el historial en `flyway_schema_history`.



LA REGLA DE ORO:
Una vez aplicada, una migración NUNCA se modifica.

Si necesitas cambiar algo, crea una nueva migración (V4). Flyway calcula checksums de los archivos aplicados; si modificas un archivo antiguo, el sistema abortará el arranque para protegerte de divergencias entre entornos.

Estrategias de Esquema: Prototipado vs Producción

Desarrollo Temprano



- **H2 Database:** Base de datos en memoria (rápida, efímera).
- `ddl-auto=update`: Hibernate infiere y modifica el esquema mágicamente basado en tus clases `@Entity`. (Útil, pero peligroso a gran escala).



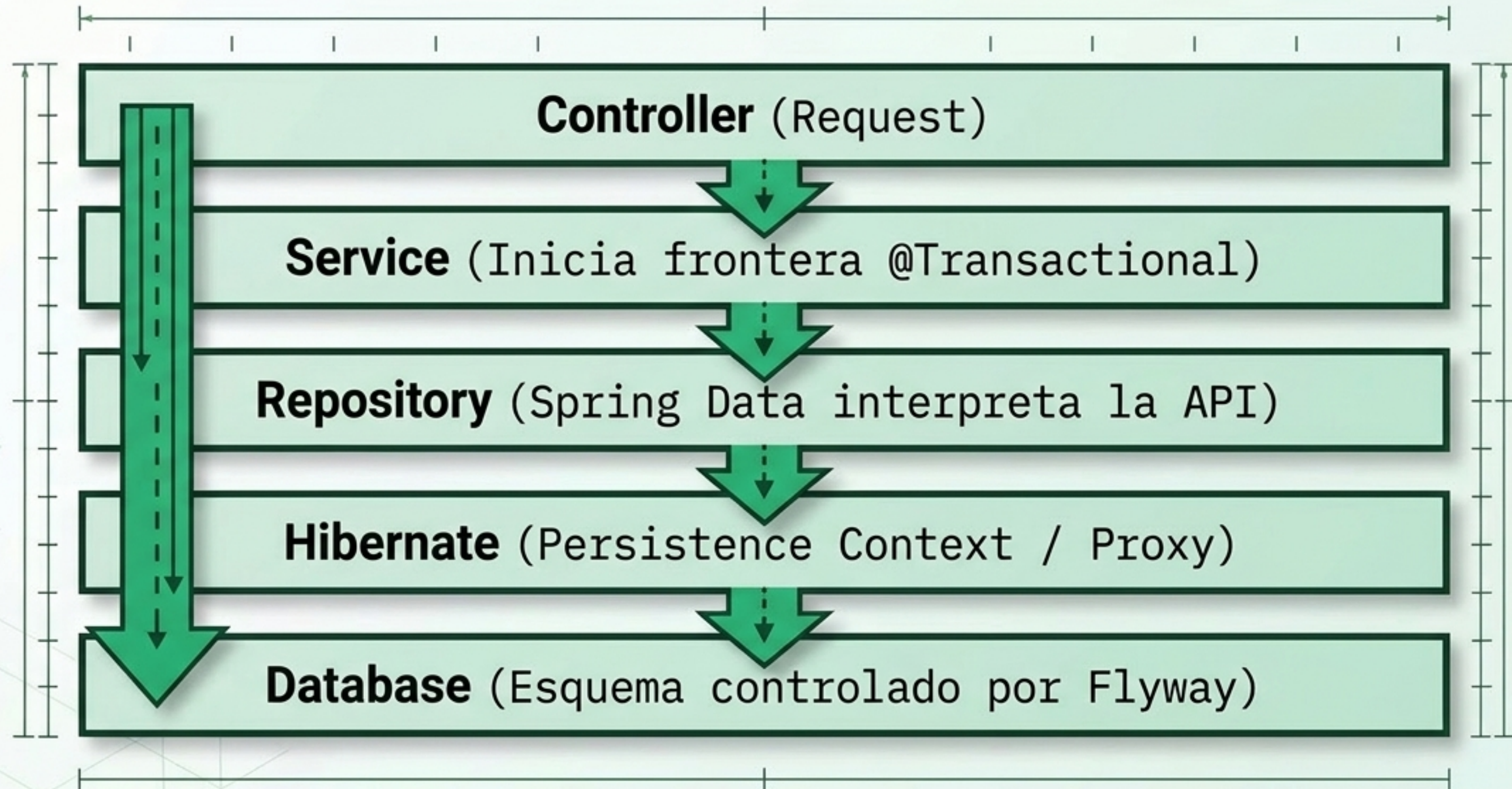
Producción Rigurosa



- **PostgreSQL / MySQL:** Motor persistente real.
- **Flyway + `ddl-auto=validate`:** Flyway es la única fuente de verdad para la estructura SQL.
- Hibernate solo verifica que el código Java coincida exactamente con el esquema SQL, fallando si hay desajustes.

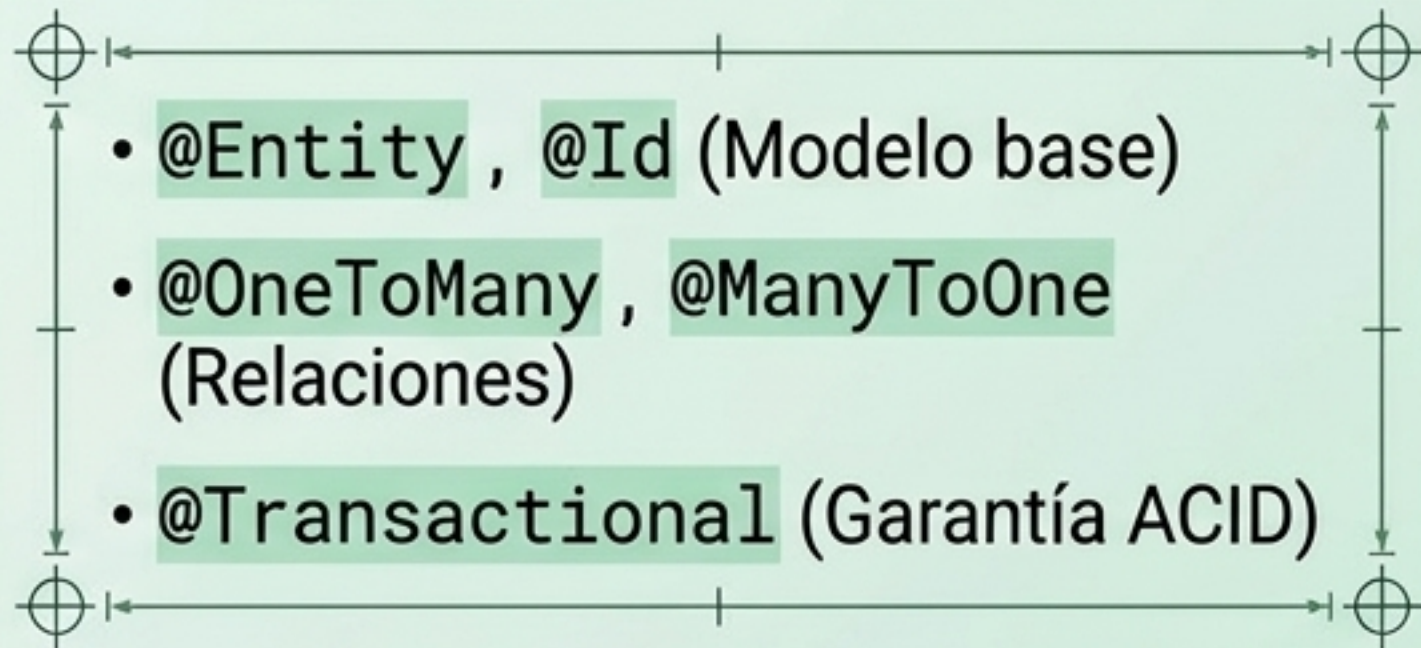
Síntesis: El Viaje Completo de los Datos

La arquitectura en capas garantiza un flujo direccional y controlado, aislando la lógica de negocio del SQL.



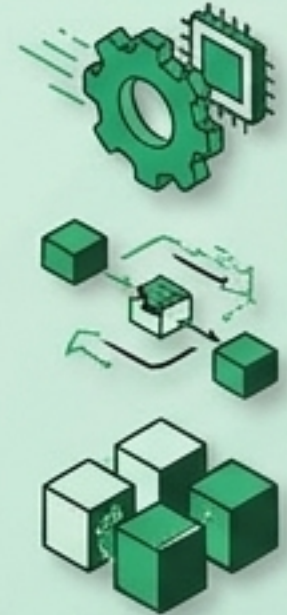
Resumen del Módulo 6

Anotaciones Clave

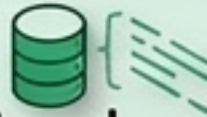


Conceptos Críticos

- **Dirty Checking:** Persistencia automática sin `save()`.
- **Problema N+1:** Evítalo con `JOIN FETCH`.
- **Propagación:** Controla transacciones anidadas.



Herramientas del Ecosistema

- **Spring Data JPA:**  Genera queries optimizadas por convención.

- **H2 Database:**  Pruebas veloces en memoria.

- **Flyway:**  Versionado inmutable de la estructura SQL.